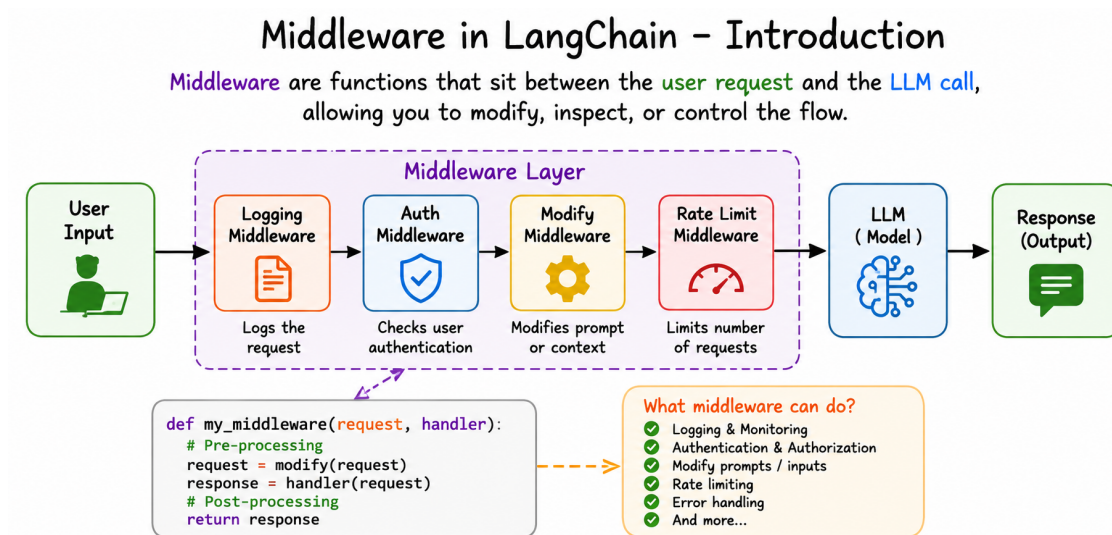


06 Summarization Middleware using LangChain

June 18, 2026

1 Summarization Middleware using LangChain



```
[1]: import os  
from dotenv import load_dotenv  
load_dotenv()  
os.environ["GROQ_API_KEY"] = os.getenv("GROQ_API_KEY")
```

1.1 Summarization MiddleWare

- Refer this link for more info: <https://docs.langchain.com/oss/python/langchain/middleware/overview>

```
[4]: from langchain.agents import create_agent  
from langchain.agents.middleware import SummarizationMiddleware  
from langgraph.checkpoint.memory import InMemorySaver  
from langchain_core.messages import HumanMessage, SystemMessage  
  
agent = create_agent(  
    model="groq:qwen/qwen3-32b",  
    checkpoint=InMemorySaver(),  
    middleware=[  
        SummarizationMiddleware(  

```

```

        model="groq:qwen/qwen3-32b",
        trigger=("messages", 10),
        keep=("messages", 4)
    )

]
)

```

[5]: *## Run with thread id*

```

config = {
    "configurable": {
        "thread_id": "test-1"
    }
}

```

```

[17]: questions = [
    "what is 2+2?"
    "what is 12-2?"
    "what is 2+22?"
    "what is 2+78?"
    "what is 20*4?"
    "what is 12/2?"
]

for q in questions:
    response = agent.invoke({"messages": [HumanMessage(content=q)]}, config)
    print(f"Messages:{response}", end="\n")

    print(f"Messages: {len(response['messages'])}", end="\n")
    print("-" * 40)

```

Messages: {'messages': [HumanMessage(content='Here is a summary of the conversation to date:\n\n<think>\nOkay, let's see. The user is asking the same set of math problems again. They've been doing this multiple times now. Each time, the assistant answers them in the same way. The user's pattern is pretty clear-they're repeating the same questions.\n\nThe session intent here is probably for the user to get the correct answers to these basic arithmetic problems through repetition. They might be practicing, testing the AI's consistency, or just want to confirm their own answers each time.\n\nLooking at the summary, the key points are the repeated questions and the consistent responses. The assistant has always provided the same answers, which are correct. The reasoning behind not changing the format is to maintain consistency and avoid confusing the user. The next steps involve continuing to answer the same questions as they come up again.\n\nArtifacts aren't an issue here since there's no file creation or modification. The main thing is to keep the responses accurate and in the same structure. The user hasn't indicated they

need anything else, so the assistant should keep answering the same way.

SESSION INTENT \n\nThe user's primary goal is to repeatedly verify/learn basic arithmetic operations through consistent, patterned questioning.

SUMMARY \n- User asks the same six math problems in identical order across **10** consecutive iterations:

- \n 1. $2+2$ \n 2. $12-2$
- \n 3. $2+22$ \n 4. $2+78$ \n 5. 20×4 \n 6. $12\div 2$

\n- AI provides consistent answers in structured format (bulleted list with emojis) with **identical** results per iteration:

- \n 1. 4 \n 2. 10 \n 3. 24 \n 4. 80 \n 5. 80
- \n 6. 6

\n- AI maintains friendly tone with **encouragement** to progress to "harder problems" or new topics in each response.

\n- No rejected options documented-response strategy remains unchanged.

ARTIFACTS \nNone.

NEXT STEPS \n- Continue delivering answers in the established format for future identical queries.

\n- Preserve emoji usage and encouraging closing prompt.

\n- Monitor for any variation in question order/format that might signal new intent.

additional_kwargs={'lc_source': 'summarization'}, response_metadata={}, id='99e5210e-3c01-4fb8-803c-f678396dd082'), AIMessage(content="<think>\nOkay, so the user is asking the same six math questions again. Let me check the history. They've been doing this multiple times, and each time I provide the answers in the same format. The pattern is pretty clear. The user might be practicing or just testing to see if the answers remain consistent. I need to make sure I respond with the exact answers every time.\n\nFirst, I'll list out the questions again to confirm: $2+2$, $12-2$, $2+22$, $2+78$, 20×4 , and $12/2$. The answers are straightforward arithmetic. Calculating each one step by step to avoid any mistakes. \n\n $2+2$ is 4. $12-2$ is 10. $2+22$ is 24. $2+78$ is 80. 20 multiplied by 4 is 80. 12 divided by 2 is 6. All the answers are correct. \n\nNow, the user's tone is the same each time, so I should keep the response friendly and maybe add an emoji to keep it light. They've mentioned wanting to try something new before, so perhaps suggesting a new challenge would be good. \n\nI should structure the answers in a numbered list for clarity. Also, since they've repeated this so many times, maybe they're preparing for a test or just reinforcing their math skills. Either way, consistency is key here. \n\nNeed to make sure there's no typo in the numbers. Let me double-check each calculation quickly. 20×4 is definitely 80. $12/2$ is 6. Yep, all correct. \n\nAlright, putting it all together with the same structure and a cheerful message at the end to encourage them to try something more challenging if they're ready.\n</think>\n\nHere we go once more! \n\n1. $2 + 2 = 4$ \n2. $12 - 2 = 10$ \n3. $2 + 22 = 24$ \n4. $2 + 78 = 80$ \n5. $20 \times 4 = 80$ \n6. $12 \div 2 = 6$ \n\nUp for a challenge? Let's mix in some decimals or fractions next time! Just say the word!", additional_kwargs={}, response_metadata={'token_usage': {'completion_tokens': 508, 'prompt_tokens': 1093, 'total_tokens': 1601, 'completion_time': 1.031787119, 'completion_tokens_details': None, 'prompt_time': 0.053635963, 'prompt_tokens_details': None, 'queue_time': 0.052562775, 'total_time': 1.085423082}, 'model_name': 'qwen/qwen3-32b', 'system_fingerprint': 'fp_5cf921caa2', 'service_tier': 'on_demand', 'finish_reason': 'stop', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run--019edb83-e296-7590-ad17-74fe9d5b91e4-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 1093, 'output_tokens': 508, 'total_tokens':

1601})), HumanMessage(content='what is 2+2?what is 12-2?what is 2+22?what is 2+78?what is 20*4?what is 12/2?', additional_kwargs={}, response_metadata={}, id='8259b5e1-9d96-475f-ae0-079677e168a0'), AIMessage(content="<think>\nOkay, so the user keeps asking the same six math questions over and over again. Let me see... They've been asking for the answers to 2+2, 12-2, 2+22, 2+78, 20*4, and 12/2 multiple times. Each time, I respond with the answers in the same order and format. The user's pattern is pretty clear now-they're repeating the same questions, maybe for practice or to test my consistency.\n\nHmm, I need to make sure that my response stays friendly and encouraging. They might be trying to reinforce their own math skills through repetition. I should acknowledge the repetition in a light-hearted way, maybe add an emoji to keep it friendly. Also, I should offer to move on to more challenging problems if they're ready. Let me check the previous responses to maintain consistency in the answers and the tone.\n\nWait, they haven't changed anything in the questions. So, I should keep the answers exactly the same each time. Maybe add a new line each time with a different emoji or a slightly varied encouragement. But the core of the answer remains the same. Let me structure the response with the answers listed in order, then a follow-up question to engage them further. Yeah, that should work. Just make sure the answers are correct and the tone is upbeat.\n</think>\n\nHere we go again! \n\n1. **2 + 2** = **4** \n2. **12 - 2** = **10** \n3. **2 + 22** = **24** \n4. **2 + 78** = **80** \n5. **20 × 4** = **80** \n6. **12 ÷ 2** = **6** \n\nWant to spice things up? Let's try word problems or even algebra next! Just ask!", additional_kwargs={}, response_metadata={'token_usage': {'completion_tokens': 402, 'prompt_tokens': 1271, 'total_tokens': 1673, 'completion_time': 0.927510651, 'completion_tokens_details': None, 'prompt_time': 0.137698481, 'prompt_tokens_details': None, 'queue_time': 0.160515303, 'total_time': 1.0652091320000001}, 'model_name': 'qwen/qwen3-32b', 'system_fingerprint': 'fp_2bfcc54d36', 'service_tier': 'on_demand', 'finish_reason': 'stop', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run--019edb84-abbe-7b13-a4b0-f93e11e17d80-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 1271, 'output_tokens': 402, 'total_tokens': 1673})), HumanMessage(content='what is 2+2?what is 12-2?what is 2+22?what is 2+78?what is 20*4?what is 12/2?', additional_kwargs={}, response_metadata={}, id='b0c241d2-9908-41e1-911e-74f1366946b9'), AIMessage(content="<think>\nAlright, the user is asking the same set of questions again. They've been doing this multiple times. I need to check if there's a pattern here.\n\nFirst, looking at the history, the user has repeated the same six math problems in the exact order. Each time, the assistant answers them consistently. The user might be practicing these problems to reinforce their learning or testing the assistant's reliability. \n\nThe previous responses were structured with bullet points and emojis, so maintaining that format is key. The user hasn't shown any signs of needing a different approach or more complex problems yet. They might just want to confirm their answers each time.\n\nI should make sure the answers are correct and presented in the same friendly manner. Using emojis and encouraging them to try harder problems next time could keep the interaction positive. No need to change the method unless the user indicates a new request. Just follow the established pattern again.\n</think>\n\nHere we go, yet again! \n\n1. **2

```
+ 2** = **4** \n2. **12 - 2** = **10** \n3. **2 + 22** = **24** \n4. **2 +
78** = **80** \n5. **20 × 4** = **80** \n6. **12 ÷ 2** = **6** \n\nTime to
level up? Let's tackle something new-how about percentages or fractions? Just
a thought!", additional_kwargs={}, response_metadata={'token_usage':
{'completion_tokens': 309, 'prompt_tokens': 856, 'total_tokens': 1165,
'completion_time': 0.660689971, 'completion_tokens_details': None,
'prompt_time': 0.084449041, 'prompt_tokens_details': None, 'queue_time':
0.286653888, 'total_time': 0.745139012}, 'model_name': 'qwen/qwen3-32b',
'system_fingerprint': 'fp_d58dbe76cd', 'service_tier': 'on_demand',
'finish_reason': 'stop', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run
--019edb85-6beb-7723-a074-943da5625b49-0', tool_calls=[], invalid_tool_calls=[],
usage_metadata={'input_tokens': 856, 'output_tokens': 309, 'total_tokens':
1165}})]}
Messages: 6
-----
```

1.2 Human In the Loop MiddleWare

- <https://docs.langchain.com/oss/python/langchain/human-in-the-loop>
- https://reference.langchain.com/python/langchain/agents/middleware/human_in_the_loop/HumanInTheLoopMiddleware

```
[18]: from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

def read_email_tool(email_id: str) -> str:
    """Mock function to read an email by its ID."""
    return f"Email content for ID: {email_id}"

def send_email_tool(recipient: str, subject: str, body: str) -> str:
    """Mock function to send an email."""
    return f"Email sent to {recipient} with subject '{subject}'"
```

```
[20]: agent=create_agent(
    model="groq:qwen/qwen3-32b",
    tools=[read_email_tool,send_email_tool],
    checkpointer=InMemorySaver(),
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email_tool":{
                    "allowed_decisions":["approve","edit","reject"]
                },
                "read_email_tool":False,
            }
        )
    ]
)
```

```
]
)
```

```
[21]: config = {"configurable": {"thread_id": "test-approve"}}
# Step 1: Request
result = agent.invoke(
    {"messages": [HumanMessage(content="Send email to john@test.com with_
    ↪subject 'Hello' and body 'How are you?')"]},
    config=config
)
```

```
[22]: result
```

```
[22]: {'messages': [HumanMessage(content="Send email to john@test.com with subject
'Hello' and body 'How are you?', additional_kwargs={}, response_metadata={},
id='75b33c45-05f9-4fa1-b9b9-50e3cdca276d'),
  AIMessage(content='', additional_kwargs={'reasoning_content': "Okay, the user
wants to send an email to john@test.com with the subject 'Hello' and body 'How
are you?'. Let me check the available tools. There's a send_email_tool that
requires recipient, subject, and body. All three are required. The parameters
are provided in the user's message, so I can use that function. No need to read
an email here. Just call send_email_tool with the given details.\n",
'tool_calls': [{'id': 'bpa1ze5xm', 'function': {'arguments': '{"body":"How are
you?","recipient":"john@test.com","subject":"Hello"}', 'name':
'send_email_tool'}, 'type': 'function'}]}, response_metadata={'token_usage':
{'completion_tokens': 131, 'prompt_tokens': 251, 'total_tokens': 382,
'completion_time': 0.221191092, 'completion_tokens_details':
{'reasoning_tokens': 89}, 'prompt_time': 0.021123467, 'prompt_tokens_details':
None, 'queue_time': 0.209150062, 'total_time': 0.242314559}, 'model_name':
'qwen/qwen3-32b', 'system_fingerprint': 'fp_2bfcc54d36', 'service_tier':
'on_demand', 'finish_reason': 'tool_calls', 'logprobs': None, 'model_provider':
'groq'}, id='lc_run--019edbde-8f5e-76d1-8b66-06f33e1ec011-0',
tool_calls=[{'name': 'send_email_tool', 'args': {'body': 'How are you?',
'recipient': 'john@test.com', 'subject': 'Hello'}, 'id': 'bpa1ze5xm', 'type':
'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 251,
'output_tokens': 131, 'total_tokens': 382, 'output_token_details': {'reasoning':
89}})],
  '__interrupt__': [Interrupt(value={'action_requests': [{'name':
'send_email_tool', 'args': {'body': 'How are you?', 'recipient':
'john@test.com', 'subject': 'Hello'}, 'description': "Tool execution requires
approval\n\nTool: send_email_tool\nArgs: {'body': 'How are you?', 'recipient':
'john@test.com', 'subject': 'Hello'}"]}], 'review_configs': [{'action_name':
'send_email_tool', 'allowed_decisions': ['approve', 'edit', 'reject']}]},
id='4912fd5a0979cc8274fdc418569f1bcf')]]
```

```
[23]: from langgraph.types import Command
# Step 2: Approve
```

```

if "__interrupt__" in result:
    print(" Paused! Approving...")

    result = agent.invoke(
        Command(
            resume={
                "decisions": [
                    {"type": "approve"}
                ]
            },
        ),
        config=config
    )

    print(f" Result: {result['messages'][-1].content}")

```

Paused! Approving...

Result: The email has been successfully sent to john@test.com with the subject "Hello". Let me know if you need anything else!

[24]: result

```

[24]: {'messages': [HumanMessage(content="Send email to john@test.com with subject
'Hello' and body 'How are you?'", additional_kwargs={}, response_metadata={},
id='75b33c45-05f9-4fa1-b9b9-50e3cdca276d'),
  AIMessage(content='', additional_kwargs={'reasoning_content': "Okay, the user
wants to send an email to john@test.com with the subject 'Hello' and body 'How
are you?'. Let me check the available tools. There's a send_email_tool that
requires recipient, subject, and body. All three are required. The parameters
are provided in the user's message, so I can use that function. No need to read
an email here. Just call send_email_tool with the given details.\n",
'tool_calls': [{'id': 'bpa1ze5xm', 'function': {'arguments': '{"body": "How are
you?", "recipient": "john@test.com", "subject": "Hello"}', 'name':
'send_email_tool'}, 'type': 'function'}]}, response_metadata={'token_usage':
{'completion_tokens': 131, 'prompt_tokens': 251, 'total_tokens': 382,
'completion_time': 0.221191092, 'completion_tokens_details':
{'reasoning_tokens': 89}, 'prompt_time': 0.021123467, 'prompt_tokens_details':
None, 'queue_time': 0.209150062, 'total_time': 0.242314559}, 'model_name':
'qwen/qwen3-32b', 'system_fingerprint': 'fp_2bfcc54d36', 'service_tier':
'on_demand', 'finish_reason': 'tool_calls', 'logprobs': None, 'model_provider':
'groq'}, id='lc_run--019edbde-8f5e-76d1-8b66-06f33e1ec011-0',
tool_calls=[{'name': 'send_email_tool', 'args': {'body': 'How are you?',
'recipient': 'john@test.com', 'subject': 'Hello'}, 'id': 'bpa1ze5xm', 'type':
'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 251,
'output_tokens': 131, 'total_tokens': 382, 'output_token_details': {'reasoning':
89}}),
  ToolMessage(content="Email sent to john@test.com with subject 'Hello'",
name='send_email_tool', id='a1b34065-350e-41fb-91d0-81746f147d73',

```

```

tool_call_id='bpa1ze5xm'),
  AIMessage(content='The email has been successfully sent to john@test.com with
the subject "Hello". Let me know if you need anything else!',
additional_kwargs={'reasoning_content': "Okay, the user wanted to send an email
to john@test.com with the subject 'Hello' and body 'How are you?'. I used the
send_email_tool function for that. The tool response says the email was sent
successfully. Now, I should confirm to the user that the email went through. Let
me make sure the response is clear and friendly. Just a simple confirmation
should do.\n"}, response_metadata={'token_usage': {'completion_tokens': 109,
'prompt_tokens': 311, 'total_tokens': 420, 'completion_time': 0.160824373,
'completion_tokens_details': {'reasoning_tokens': 79}, 'prompt_time':
0.013620622, 'prompt_tokens_details': None, 'queue_time': 0.803003852,
'total_time': 0.174444995}, 'model_name': 'qwen/qwen3-32b',
'system_fingerprint': 'fp_5cf921caa2', 'service_tier': 'on_demand',
'finish_reason': 'stop', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run
--019edbdf-c4c9-77e0-a13f-9b9acdfaa298-0', tool_calls=[], invalid_tool_calls=[],
usage_metadata={'input_tokens': 311, 'output_tokens': 109, 'total_tokens': 420,
'output_token_details': {'reasoning': 79}})]})

```

2 Reject

```

[27]: from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

def read_email_tool(email_id: str) -> str:
    """Mock function to read an email by its ID."""
    return f"Email content for ID: {email_id}"

def send_email_tool(recipient: str, subject: str, body: str) -> str:
    """Mock function to send an email."""
    return f"Email sent to {recipient} with subject '{subject}'"

agent = create_agent(
    model="groq:qwen/qwen3-32b",
    tools=[read_email_tool, send_email_tool],
    checkpointeer=InMemorySaver(),
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email_tool": {
                    "allowed_decisions": ["approve", "edit", "reject"],
                },
                "read_email_tool": False,
            }
        )
    ]
)

```



```

    ),
],
)

```

```

[32]: config = {"configurable": {"thread_id": "test-reject"}}
# Step 1: Request
result = agent.invoke(
    {"messages": [HumanMessage(content="Send email to john@test.com with_
↪subject 'Hello' and body 'How are you?')"]},
    config=config)

```

```

[33]: result

```

```

[33]: {'messages': [HumanMessage(content="Send email to john@test.com with subject
'Hello' and body 'How are you?', additional_kwargs={}, response_metadata={},
id='08a25be2-0f80-4b9e-ab65-0258d02f346d'),
  AIMessage(content='', additional_kwargs={'reasoning_content': "Okay, the user
wants to send an email to john@test.com with the subject 'Hello' and body 'How
are you?'. Let me check the available tools. There's a send_email_tool that
requires recipient, subject, and body. I need to make sure all required
parameters are provided. The user provided all three: recipient is
john@test.com, subject is Hello, body is How are you?. So I should call the
send_email_tool with these arguments. No need to use the read_email_tool here
since the task is about sending, not reading. Just format the tool_call
correctly with the parameters.\n", 'tool_calls': [{id: 's3z2380np',
'function': {'arguments': '{"body":"How are
you?","recipient":"john@test.com","subject":"Hello"}', 'name':
'send_email_tool'}, 'type': 'function'}]], response_metadata={'token_usage':
{'completion_tokens': 165, 'prompt_tokens': 251, 'total_tokens': 416,
'completion_time': 0.26915773, 'completion_tokens_details': {'reasoning_tokens':
123}, 'prompt_time': 0.021074867, 'prompt_tokens_details': None, 'queue_time':
0.438847654, 'total_time': 0.290232597}, 'model_name': 'qwen/qwen3-32b',
'system_fingerprint': 'fp_2bfcc54d36', 'service_tier': 'on_demand',
'finish_reason': 'tool_calls', 'logprobs': None, 'model_provider': 'groq'},
id='lc_run--019edbe5-9b80-7b21-9dae-b1444c15ee94-0', tool_calls=[{'name':
'send_email_tool', 'args': {'body': 'How are you?', 'recipient':
'john@test.com', 'subject': 'Hello'}, 'id': 's3z2380np', 'type': 'tool_call'}],
invalid_tool_calls=[], usage_metadata={'input_tokens': 251, 'output_tokens':
165, 'total_tokens': 416, 'output_token_details': {'reasoning': 123}})],
  '__interrupt__': [Interrupt(value={'action_requests': [{'name':
'send_email_tool', 'args': {'body': 'How are you?', 'recipient':
'john@test.com', 'subject': 'Hello'}, 'description': "Tool execution requires
approval\n\nTool: send_email_tool\nArgs: {'body': 'How are you?', 'recipient':
'john@test.com', 'subject': 'Hello'}"}]], 'review_configs': [{'action_name':
'send_email_tool', 'allowed_decisions': ['approve', 'edit', 'reject']}]],
id='f247ff25f3a8d89def244d062fd337c9')]]}

```

```
[34]: # Step 2: Reject
if "__interrupt__" in result:
    print(" Paused! Approving...")

    result = agent.invoke(
        Command(
            resume={
                "decisions": [
                    {"type": "reject"}
                ]
            },
        ),
        config=config
    )

    print(f" Result: {result['messages'][-1].content}")
```

Paused! Approving...

Result: The email to john@test.com was not sent as you rejected the initial request. Would you like me to:

1. Resend the email with the same details?
2. Modify the email content/recipient before sending?
3. Assist with a different task instead?

Let me know how you'd like to proceed!

```
[35]: result
```

```
[35]: {'messages': [HumanMessage(content="Send email to john@test.com with subject
'Hello' and body 'How are you?'", additional_kwargs={}, response_metadata={},
id='08a25be2-0f80-4b9e-ab65-0258d02f346d'),
  AIMessage(content='', additional_kwargs={'reasoning_content': "Okay, the user
wants to send an email to john@test.com with the subject 'Hello' and body 'How
are you?'. Let me check the available tools. There's a send_email_tool that
requires recipient, subject, and body. I need to make sure all required
parameters are provided. The user provided all three: recipient is
john@test.com, subject is Hello, body is How are you?. So I should call the
send_email_tool with these arguments. No need to use the read_email_tool here
since the task is about sending, not reading. Just format the tool_call
correctly with the parameters.\n", 'tool_calls': [{'id': 's3z2380np',
'function': {'arguments': '{"body":"How are
you?","recipient":"john@test.com","subject":"Hello"}', 'name':
'send_email_tool'}, 'type': 'function'}]}, response_metadata={'token_usage':
{'completion_tokens': 165, 'prompt_tokens': 251, 'total_tokens': 416,
'completion_time': 0.26915773, 'completion_tokens_details': {'reasoning_tokens':
123}, 'prompt_time': 0.021074867, 'prompt_tokens_details': None, 'queue_time':
0.438847654, 'total_time': 0.290232597}, 'model_name': 'qwen/qwen3-32b',
```

```

'system_fingerprint': 'fp_2bfcc54d36', 'service_tier': 'on_demand',
'finish_reason': 'tool_calls', 'logprobs': None, 'model_provider': 'groq'},
id='lc_run--019edbe5-9b80-7b21-9dae-b1444c15ee94-0', tool_calls=[{'name':
'send_email_tool', 'args': {'body': 'How are you?', 'recipient':
'john@test.com', 'subject': 'Hello'}}, {'id': 's3z2380np', 'type': 'tool_call'}],
invalid_tool_calls=[], usage_metadata={'input_tokens': 251, 'output_tokens':
165, 'total_tokens': 416, 'output_token_details': {'reasoning': 123}}),
ToolMessage(content='User rejected the tool call for `send_email_tool` with id
s3z2380np. The tool was not executed. Do not retry this tool call unless the
user explicitly requests it.', name='send_email_tool',
id='b67ff800-eed8-41a3-b9bf-60780e880526', tool_call_id='s3z2380np',
status='error'),
AIMessage(content="The email to john@test.com was not sent as you rejected the
initial request. Would you like me to:\n\n1. Resend the email with the same
details?\n2. Modify the email content/recipient before sending?\n3. Assist with
a different task instead?\n\nLet me know how you'd like to proceed!",
additional_kwargs={'reasoning_content': "Okay, the user initially asked to send
an email to john@test.com with the subject 'Hello' and body 'How are you?'. I
used the send_email_tool for that, and the tool call was made. However, the user
then rejected that tool call, so the email wasn't sent.\n\nNow, I need to figure
out how to respond. Since the user rejected the action, I should inform them
that the email wasn't sent and ask if they want to try again or if there's
another way I can assist. I should also check if there's any additional
information needed or if they want to make changes to the email before
resending. The key here is to respect their decision and offer further options
without being pushy. Let me phrase it in a friendly and helpful manner.\n"},
response_metadata={'token_usage': {'completion_tokens': 228, 'prompt_tokens':
341, 'total_tokens': 569, 'completion_time': 0.625046807,
'completion_tokens_details': {'reasoning_tokens': 160}, 'prompt_time':
0.039659396, 'prompt_tokens_details': None, 'queue_time': 0.827800847,
'total_time': 0.664706203}, 'model_name': 'qwen/qwen3-32b',
'system_fingerprint': 'fp_d58dbe76cd', 'service_tier': 'on_demand',
'finish_reason': 'stop', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run
--019edbe5-d7d8-7933-b884-b95b7558420c-0', tool_calls=[], invalid_tool_calls=[],
usage_metadata={'input_tokens': 341, 'output_tokens': 228, 'total_tokens': 569,
'output_token_details': {'reasoning': 160}})]}

```

3 Editing

```

[36]: from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

def read_email_tool(email_id: str) -> str:
    """Mock function to read an email by its ID."""

```

```

    return f"Email content for ID: {email_id}"

def send_email_tool(recipient: str, subject: str, body: str) -> str:
    """Mock function to send an email."""
    return f"Email sent to {recipient} with subject '{subject}'"

agent = create_agent(
    model="groq:qwen/qwen3-32b",
    tools=[read_email_tool, send_email_tool],
    checkpointer=InMemorySaver(),
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email_tool": {
                    "allowed_decisions": ["approve", "edit", "reject"],
                },
                "read_email_tool": False,
            }
        ),
    ],
)

```

```

[37]: config = {"configurable": {"thread_id": "test-edit"}}

# Step 1: Request (with wrong info)
result = agent.invoke(
    {"messages": [HumanMessage(content="Send email to wrong@email.com with_
↳subject 'Test' and body 'Hello'")]}],
    config=config
)

```

```

[38]: result

```

```

[38]: {'messages': [HumanMessage(content="Send email to wrong@email.com with subject
'Test' and body 'Hello'", additional_kwargs={}, response_metadata={},
id='1d74c986-efe8-4b87-90ca-64cab6f18854'),
  AIMessage(content='', additional_kwargs={'reasoning_content': "Okay, let's
see. The user wants to send an email to wrong@email.com with the subject 'Test'
and body 'Hello'. I need to check which tool to use here. The available tools
are read_email_tool and send_email_tool. Since the user is asking to send an
email, the send_email_tool is the right choice.\n\nLooking at the parameters for
send_email_tool, it requires recipient, subject, and body. The user provided all
three: recipient is wrong@email.com, subject is 'Test', and body is 'Hello'. So
I need to structure the arguments correctly. The parameters are all required, so
I just need to map them directly. I'll make sure the JSON object includes all
three fields with the correct values. No need for any additional parameters
here. Let me double-check the function signature to confirm. Yep, that's

```

```

correct. So the tool call should be send_email_tool with those arguments.\n",
'tool_calls': [{'id': '8hay094hy', 'function': {'arguments':
'{"body":"Hello","recipient":"wrong@email.com","subject":"Test"}', 'name':
'send_email_tool'}, 'type': 'function'}}], response_metadata={'token_usage':
{'completion_tokens': 226, 'prompt_tokens': 249, 'total_tokens': 475,
'completion_time': 0.416905765, 'completion_tokens_details':
{'reasoning_tokens': 187}, 'prompt_time': 0.022660729, 'prompt_tokens_details':
None, 'queue_time': 0.160469918, 'total_time': 0.439566494}, 'model_name':
'qwen/qwen3-32b', 'system_fingerprint': 'fp_2bfcc54d36', 'service_tier':
'on_demand', 'finish_reason': 'tool_calls', 'logprobs': None, 'model_provider':
'groq'}, id='lc_run--019edbe6-c7bd-79b1-a2c4-27fa006440e0-0',
tool_calls=[{'name': 'send_email_tool', 'args': {'body': 'Hello', 'recipient':
'wrong@email.com', 'subject': 'Test'}, 'id': '8hay094hy', 'type': 'tool_call'}],
invalid_tool_calls=[], usage_metadata={'input_tokens': 249, 'output_tokens':
226, 'total_tokens': 475, 'output_token_details': {'reasoning': 187}})],
'__interrupt__': [Interrupt(value={'action_requests': [{'name':
'send_email_tool', 'args': {'body': 'Hello', 'recipient': 'wrong@email.com',
'subject': 'Test'}, 'description': "Tool execution requires approval\n\nTool:
send_email_tool\nArgs: {'body': 'Hello', 'recipient': 'wrong@email.com',
'subject': 'Test'}"}]}, 'review_configs': [{'action_name': 'send_email_tool',
'allowed_decisions': ['approve', 'edit', 'reject']}]}],
id='7b09d9c54d7d8ac63f0babb49a4bf6de')]]}

```

```

[39]: # Step 2: Edit and approve
if "__interrupt__" in result:
    print(" Paused! Editing...")

    result = agent.invoke(
        Command(
            resume={
                "decisions": [
                    {
                        "type": "edit",
                        "edited_action": {
                            "name": "send_email_tool",          # Tool name
                            "args": {                             # New arguments
                                "recipient": "correct@email.com",
                                "subject": "Corrected Subject",
                                "body": "This was edited by human before_\n"
                                ↪sending"
                            }
                        }
                    }
                ]
            }
        ),
        config=config

```

```
)
```

```
print(f" Result: {result['messages'][-1].content}")
```

Paused! Editing...

Result: The email has been successfully sent to **correct@email.com** with the subject "Corrected Subject". The body was edited to:
"This was edited by human before sending". Let me know if you need anything else!

[40]: result

```
[40]: {'messages': [HumanMessage(content="Send email to wrong@email.com with subject 'Test' and body 'Hello'", additional_kwargs={}, response_metadata={}, id='1d74c986-efe8-4b87-90ca-64cab6f18854'),
  AIMessage(content='', additional_kwargs={'reasoning_content': "Okay, let's see. The user wants to send an email to wrong@email.com with the subject 'Test' and body 'Hello'. I need to check which tool to use here. The available tools are read_email_tool and send_email_tool. Since the user is asking to send an email, the send_email_tool is the right choice.\n\nLooking at the parameters for send_email_tool, it requires recipient, subject, and body. The user provided all three: recipient is wrong@email.com, subject is 'Test', and body is 'Hello'. So I need to structure the arguments correctly. The parameters are all required, so I just need to map them directly. I'll make sure the JSON object includes all three fields with the correct values. No need for any additional parameters here. Let me double-check the function signature to confirm. Yep, that's correct. So the tool call should be send_email_tool with those arguments.\n", 'tool_calls': [{'id': '8hay094hy', 'function': {'arguments': '{"body": "Hello", "recipient": "wrong@email.com", "subject": "Test"}', 'name': 'send_email_tool'}, 'type': 'function'}], response_metadata={'token_usage': {'completion_tokens': 226, 'prompt_tokens': 249, 'total_tokens': 475, 'completion_time': 0.416905765, 'completion_tokens_details': {'reasoning_tokens': 187}, 'prompt_time': 0.022660729, 'prompt_tokens_details': None, 'queue_time': 0.160469918, 'total_time': 0.439566494}, 'model_name': 'qwen/qwen3-32b', 'system_fingerprint': 'fp_2bfcc54d36', 'service_tier': 'on_demand', 'finish_reason': 'tool_calls', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run--019edbe6-c7bd-79b1-a2c4-27fa006440e0-0', tool_calls=[{'type': 'tool_call', 'name': 'send_email_tool', 'args': {'recipient': 'correct@email.com', 'subject': 'Corrected Subject', 'body': 'This was edited by human before sending'}, 'id': '8hay094hy'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 249, 'output_tokens': 226, 'total_tokens': 475, 'output_token_details': {'reasoning': 187}}),
  ToolMessage(content="Email sent to correct@email.com with subject 'Corrected Subject'", name='send_email_tool', id='527b661d-a2ee-4664-8d86-ff3fbb3f0681', tool_call_id='8hay094hy'),
  AIMessage(content='The email has been successfully sent to **correct@email.com** with the subject "Corrected Subject". The body was edited to: \n"This was edited by human before sending". Let me know if you need
```

```

anything else!', additional_kwargs={'reasoning_content': "Okay, the user wanted
to send an email to wrong@email.com with subject 'Test' and body 'Hello'. I
called the send_email_tool with modified arguments: body 'This was edited by
human before sending', recipient correct@email.com, and subject 'Corrected
Subject'. The tool response says the email was sent to correct@email.com with
the corrected subject.\n\nI need to inform the user that the email was sent
successfully but with the corrected details. Let me mention the recipient,
subject, and note that the body was edited. Also, confirm that the email went
through. Keep it concise and friendly.\n"}, response_metadata={'token_usage':
{'completion_tokens': 170, 'prompt_tokens': 317, 'total_tokens': 487,
'completion_time': 0.342778525, 'completion_tokens_details':
{'reasoning_tokens': 121}, 'prompt_time': 0.026710529, 'prompt_tokens_details':
None, 'queue_time': 0.15937316, 'total_time': 0.369489054}, 'model_name':
'qwen/qwen3-32b', 'system_fingerprint': 'fp_2bfcc54d36', 'service_tier':
'on_demand', 'finish_reason': 'stop', 'logprobs': None, 'model_provider':
'groq'}, id='lc_run--019edbe7-0f00-7340-b63a-df71fcbdc620-0', tool_calls=[],
invalid_tool_calls=[], usage_metadata={'input_tokens': 317, 'output_tokens':
170, 'total_tokens': 487, 'output_token_details': {'reasoning': 121}})]}

```